STUDENT MANAGEMENT SYSTEM

Python Programming Project Technical Documentation

Author: Mohamed Wael **Language:** Python 3.x **Storage:** File & CSV I/O

1. EXECUTIVE SUMMARY & OVERVIEW

The **Student Management System (SMS)** is a robust, console-based application designed to efficiently store, manage, manipulate, and retrieve student records. Developed by **Mohamed Wael** as part of the core Python curriculum, this software demonstrates mastery over essential data structures, functional programming, string processing, and file persistence mechanisms.

2. KEY FEATURES & FUNCTIONALITIES

Core Features

- **Add New Student:** Capture ID, Name, Age, and Courses. Stores record in a central dictionary keyed by Student ID.
- **View All Records:** Displays clean, formatted profiles for all registered students.
- **Search Records:** Case-insensitive search using string manipulation (`lower()`, `startswith()`).
- **Delete Student:** Safely removes student records by unique ID.

Advanced Features & File I/O

- **Course Management:** Dynamic course updating using Python `set` operations to guarantee unique entries.
- **Data Persistence:** Automatic load on application startup and bulk save to text file (`students.txt`).
- **CSV Export:** Exports student records into structured `students_export.csv` file.
- **Enrollment Analytics:** Aggregates course metrics and counts total enrollment per subject.

3. APPLIED PYTHON CONCEPTS

DATA STRUCTURE / CONCEPT	IMPLEMENTATION DETAILS & PURPOSE
Dictionaries	Serves as the primary database holding student data with unique ID keys (e.g., `students[s_id] = {...}`).
Tuples	Stores immutable initial course profiles per student to ensure data integrity once defined.
Sets	Utilized during dynamic addition/removal of courses to automatically enforce uniqueness and avoid duplicate entries.
File I/O & CSV	Handles persistent file storage using custom delimiter parsing (`|`) and standard `csv.writer` integration.
String Manipulation	Employs string filtering, trimming, and casing methods (`strip()`, `lower()`, `split()`) for queries and input cleaning.

4. SOURCE CODE IMPLEMENTATION

Below is the complete, modular Python implementation created by **Mohamed Wael**:

```
import csv
import os

FILE_NAME = "students.txt"
CSV_FILE_NAME = "students_export.csv"

# Primary Data Store
students = {}


def load_data():
    if not os.path.exists(FILE_NAME):
        return
    try:
        with open(FILE_NAME, "r", encoding="utf-8") as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue
                parts = line.split("|")
                if len(parts) == 4:
                    s_id, name, age, courses_str = parts
                    courses = tuple(courses_str.split(",")) if courses_str else ()
                    students[s_id] = {
                        "name": name,
                        "age": int(age),
                        "courses": courses,
                    }
        print("Data loaded successfully!")
    except Exception as e:
        print(f"Error loading data: {e}")


def save_data():
    try:
        with open(FILE_NAME, "w", encoding="utf-8") as f:
            for s_id, data in students.items():
                courses_str = ",".join(data["courses"])
                f.write(f"{s_id}|{data['name']}|{data['age']}|{courses_str}
")
        print("Data saved successfully!")
    except Exception as e:
        print(f"Error saving data: {e}")


def add_student():
    s_id = input("Enter Student ID: ").strip()
    if s_id in students:
        print("Error: Student ID already exists!")
        return

    name = input("Enter Student Name: ").strip()
    try:
        age = int(input("Enter Student Age: "))
    except ValueError:
        print("Error: Please enter a valid integer for age.")
        return

    courses_input = input("Enter courses separated by commas: ")
    courses_set = {c.strip() for c in courses_input.split(",") if c.strip()}
    students[s_id] = {"name": name, "age": age, "courses": tuple(courses_set)}
    print(f"Student '{name}' added successfully!")


def view_all_students():
    if not students:
```

```
        print("No student records found.")
        return
    for s_id, data in students.items():
        courses_str = ", ".join(data["courses"]) if data["courses"] else "None"
        print(f"ID: {s_id} | Name: {data['name']} | Age: {data['age']} | Courses: ({courses_str})")


def search_student():
    query = input("Enter Student ID or Name to search: ").strip().lower()
    found = False
    for s_id, data in students.items():
        if s_id.lower().startswith(query) or query in data["name"].lower():
            courses_str = ", ".join(data["courses"])
            print(f"Found -> ID: {s_id} | Name: {data['name']} | Age: {data['age']} | Courses:
({courses_str})")
            found = True
    if not found:
        print("No matching student records found.")


def update_student():
    s_id = input("Enter Student ID to update: ").strip()
    if s_id not in students:
        print("Error: Student ID not found!")
        return
    courses_set = set(students[s_id]["courses"])
    print("
1. Add Course
2. Remove Course")
    choice = input("Select option (1-2): ").strip()
    if choice == "1":
        new_course = input("Enter course to add: ").strip()
        if new_course:
            courses_set.add(new_course)
    elif choice == "2":
        rem_course = input("Enter course to remove: ").strip()
        courses_set.discard(rem_course)
    students[s_id]["courses"] = tuple(courses_set)


def delete_student():
    s_id = input("Enter Student ID to delete: ").strip()
    if s_id in students:
        del students[s_id]
        print(f"Student ID {s_id} removed successfully.")


def export_to_csv():
    try:
        with open(CSV_FILE_NAME, "w", newline="", encoding="utf-8") as f:
            writer = csv.writer(f)
            writer.writerow(["ID", "Name", "Age", "Courses"])
            for s_id, data in students.items():
                writer.writerow([s_id, data["name"], data["age"], ", ".join(data["courses"])])
        print(f"Data exported to {CSV_FILE_NAME}")
    except Exception as e:
        print(f"Error: {e}")


def course_enrollment_counts():
    counts = {}
    for data in students.values():
        for course in data["courses"]:
            counts[course] = counts.get(course, 0) + 1
    for course, count in counts.items():
        print(f"Course: {course} | Enrolled: {count}")


def main():
    load_data()
    while True:
        print("
1. Add  2. View  3. Search  4. Update  5. Delete  6. Export CSV  7. Stats  8. Exit")
```

```
        choice = input("Choice: ").strip()
        if choice == "1": add_student()
        elif choice == "2": view_all_students()
        elif choice == "3": search_student()
        elif choice == "4": update_student()
        elif choice == "5": delete_student()
        elif choice == "6": export_to_csv()
        elif choice == "7": course_enrollment_counts()
        elif choice == "8": save_data(); break

if __name__ == "__main__":
    main()
```

5. CONCLUSION

The Student Management System successfully meets all foundational functional requirements and incorporates optional extra challenges. The modular design enables easy future updates, such as adding database integration or a graphical user interface (GUI).